UNIVERSIDADE POLITÉCNICA DO CÁVADO E DO AVE

LICENCIATURA EM ENGENHARIA DE SISTEMAS INFORMÁTICOS

TRABALHO PRÁTICO I

Integração de Sistemas de Informação

Monitorização e análise de preços de combustíveis em Portugal

Processo ETL com Pentaho Data Integration, PostgreSQL e Grafana

github.com/FrozenProduction/isi-tp1

Diogo Pereira

Nº 31513 · Regime diurno

Docentes: Luís Ferreira e Óscar Ribeiro

Ano letivo 2026/2027

Escola Superior de Tecnologia
Universidade Politécnica do Cávado e do Ave

Barcelos, outubro de 2026

Resumo

Este trabalho descreve um processo ETL completo, da recolha à visualização, aplicado aos preços de combustíveis publicados pela DGEG. A fonte é uma API pública que devolve catorze mil registos por dia, com o preço em texto português e o mesmo posto repetido uma vez por combustível. O processo transforma esse ficheiro num modelo dimensional em PostgreSQL, responde a perguntas de preço por distrito e por combustível, compara cada grupo com um preço-alvo definido em ficheiro e envia um email de resumo com os grupos abaixo do alvo. A execução documentada neste relatório processou 14 213 preços de 3 143 postos, produziu 208 grupos e identificou 2 alertas em cerca de dez segundos. O material está organizado em nove transformações e um job, com exportação em quatro formatos e um dashboard sobre a base de dados.

Palavras-chave: ETL, Pentaho Data Integration, PostgreSQL, integração de dados, dashboards.

Índice

1. Enquadramento
2. Problema
3. Estratégia utilizada
4. Transformações
5. Jobs
6. Vídeo com demonstração
7. Conclusão e trabalhos futuros
8. Referências

1. Enquadramento

Este relatório documenta o Trabalho Prático I da unidade curricular de Integração de Sistemas de Informação, do terceiro ano da Licenciatura em Engenharia de Sistemas Informáticos da Escola Superior de Tecnologia, no ano letivo 2026/2027. O trabalho foi realizado individualmente.

A plataforma escolhida foi o Pentaho Data Integration (Kettle) 10.2, por ser a ferramenta usada nas aulas e por cobrir as três fases pedidas sem obrigar a escrever código de integração próprio. A base de dados é o PostgreSQL 17, que serve de destino da carga e também de origem para a fase de transformação. O dashboard foi construído no Grafana 13.2, ligado diretamente ao modelo dimensional, e a recolha contínua ficou a cargo de um fluxo no Node-RED que publica cada recolha num broker Mosquitto local.

> **Repositório (público): github.com/FrozenProduction/isi-tp1**
> Dentro do repositório: as nove transformações e o job de orquestração, os fluxos do Node-RED, o modelo de dados em SQL, o snapshot e as exportações da execução documentada, o relatório em PDF e o dashboard do Grafana exportado.

2. Problema

Os preços dos combustíveis em Portugal são publicados posto a posto pela DGEG, mas num formato que não responde a uma pergunta simples: onde abasteço mais barato esta semana, no meu distrito? O preço chega como texto em português, por exemplo `"0,849 €"`, o mesmo posto aparece em catorze linhas, uma por combustível, a marca vem escrita de formas diferentes e não existe qualquer agregação geográfica. Sem tratamento, o ficheiro é uma lista de catorze mil linhas que não permite comparar nada.

O objetivo é construir um processo ETL que transforme esse ficheiro num modelo consultável e responda a cinco perguntas: preço médio, mínimo e máximo por distrito e combustível; desvio de cada distrito à média nacional do mesmo combustível; número de postos com preço válido em cada grupo; alerta quando um grupo fica abaixo de um preço-alvo; e um dashboard com o retrato do dia e o histórico por execução.

A pertinência vem de a fonte ser oficial e mudar todos os dias, o que obriga a que o processo seja repetível e não um trabalho de limpeza feito uma vez. A dificuldade não está no volume, que é pequeno para as ferramentas usadas, mas na regra de negócio. Os catorze combustíveis não partilham a mesma unidade: há preços por litro, como o gasóleo e o GPL, e preços por quilograma ou por metro cúbico, como o GNC e o GNL. Tratar tudo como uma só família produz um resultado errado com aparência de certo: com um alvo de 2,200 €/L aplicado a todos os grupos ficam 87 dos 208 grupos abaixo do alvo, quando o número correto é 2. É esta diferença que justifica o cuidado posto na fase de transformação.

Item da fonte	Detalhe
Endpoint	`https://precoscombustiveis.dgeg.gov.pt/api/PrecoComb/PesquisarPostos?qtdPorPagina=100000&pagina=1`
Autenticação	nenhuma, API pública
Formato	JSON
Chamadas por execução	1, com todo o catálogo numa só resposta
Registos recebidos	14 213 preços de 3 143 postos, em 14 combustíveis
Tempo de resposta medido	cerca de 3 segundos
Campos usados	`Id`, `Nome`, `Marca`, `Combustivel`, `Preco`, `Distrito`, `Municipio`, `Morada`, `Localidade`, `CodPostal`, `Latitude`, `Longitude`, `DataAtualizacao`

3. Estratégia utilizada

A arquitetura separa o mundo exterior do resto do processo (Figura 1). Só a primeira transformação toca a rede. Tudo o resto lê ficheiros ou a base de dados, o que torna o processo testável sem depender da disponibilidade da API.

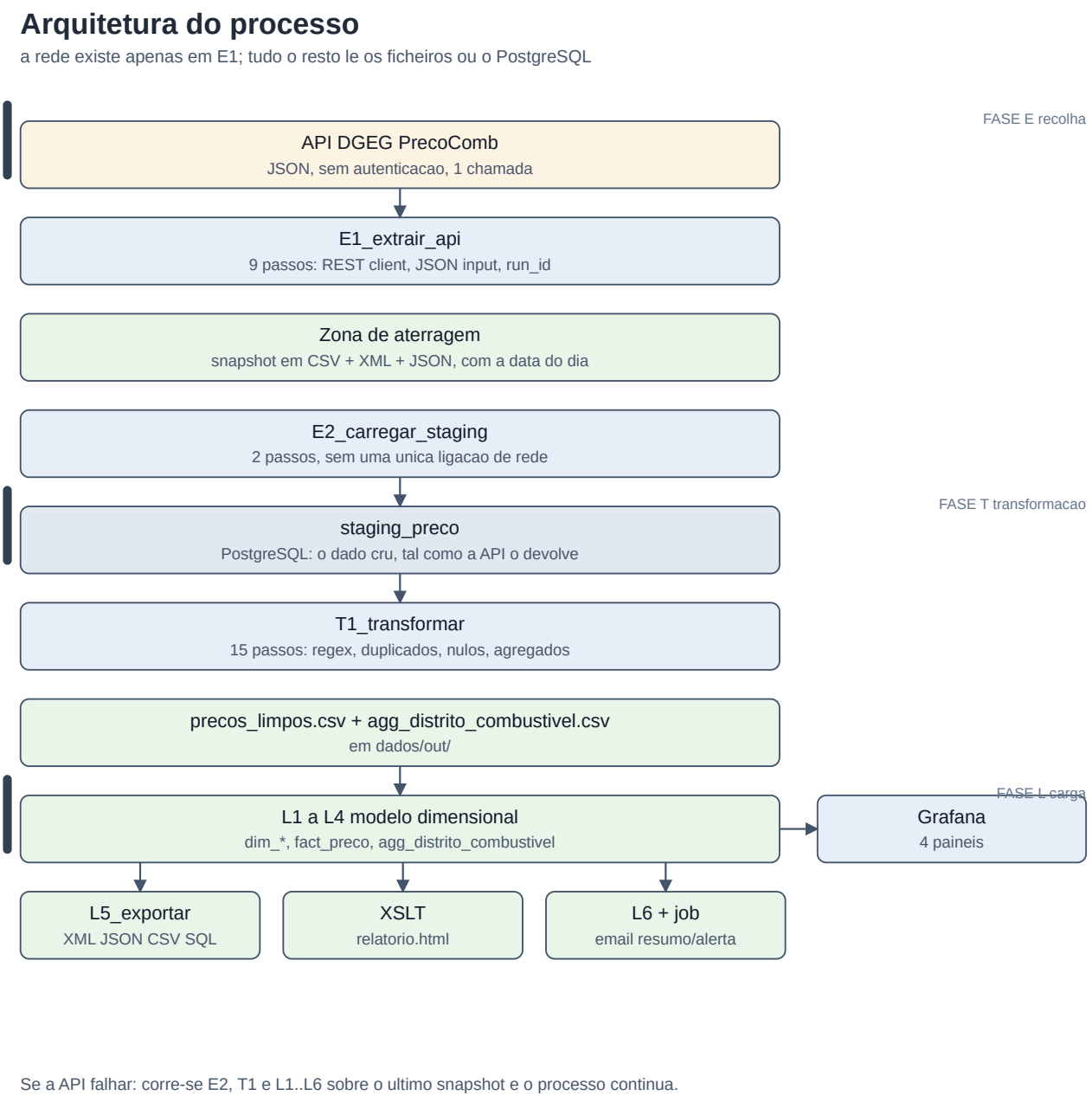


Figura 1. Arquitetura do processo: a API é contactada uma vez por execução e o resultado fica numa zona de aterragem em ficheiros datados.

A primeira decisão de desenho foi criar essa zona de aterragem, gravando o resultado da API em CSV, XML e JSON antes de qualquer transformação. Se a DGEG estiver indisponível, corre-se a segunda transformação e as seguintes sobre o último snapshot e o processo continua a produzir modelo, exportações e dashboard com o dado congelado do dia anterior. Sem esta separação, uma falha da API no dia da demonstração parava o trabalho todo.

A segunda decisão foi usar o PostgreSQL como origem e como destino. O dado cru fica guardado na tabela `staging_preco` antes de ser transformado, o que permite repetir a fase de transformação sem voltar a chamar a API e deixa registo do que foi recebido. Cada execução cria uma linha própria na tabela `execucao` e um `run_id`, e todos os registos carregados ficam associados a esse identificador. Nada é recalculado por cima do dia anterior: o modelo acumula execuções e o dashboard mostra a mais recente.

Fase	Transformações	O que produz
E Recolha e preparação	`E1_extrair_api.ktr`, `E2_carregar_staging.ktr`	snapshot em CSV, XML e JSON, e tabela `staging_preco`
T Processamento e transformação	`T1_transformar.ktr`	`precos_limpos.csv`, `agg_distrito_combustivel.csv`, `nulos.csv`, `rejeitados.csv`
L Persistência e notificação	`L1` a `L6`	dimensões, factos e agregados em PostgreSQL, quatro exportações, relatório HTML e email
Orquestração	`JOB_TP1.kjb`	uma execução completa, registada em `execucao`

Ferramenta	Versão	Função no trabalho
Pentaho Data Integration (Kettle)	10.2 CE	motor ETL: transformações e job
PostgreSQL	17	base de dados destino e, na fase T, origem
pgAdmin 4	8.x	criação e verificação do modelo
Grafana	13.2.4	dashboard sobre o modelo dimensional
Node-RED	5.0	recolha contínua da API e publicação do evento (fase E alternativa)
Mosquitto	2.1.2	broker MQTT local onde o evento é publicado
Java (Temurin)	21	runtime do Kettle
Git e GitHub		repositório com todo o material

4. Transformações

Os diagramas desta secção foram gerados a partir dos próprios ficheiros `.ktr`, lendo os passos e as ligações que lá estão definidos. Não são um desenho do que devia ser, são a malha que corre.

4.1 E1: extração da API

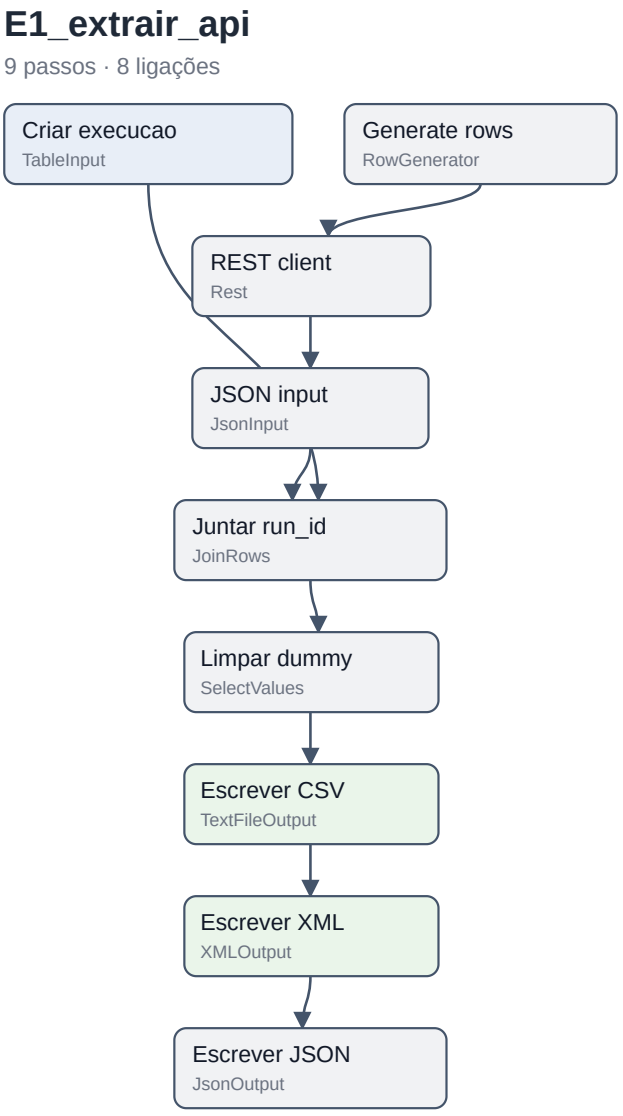


Figura 2. `E1_extrair_api.ktr`: 9 passos e 8 ligações.

Na Figura 2 vê-se a cadeia completa. O primeiro passo é uma linha gerada artificialmente. O `REST client` é um passo de lookup: faz um pedido por cada linha que recebe, e sem nada a alimentá-lo faz zero pedidos e devolve zero linhas, sem erro. Uma linha vazia resolve o problema e é também a forma como o exemplo do professor está montado.

O `JSON input` lê o campo `resultado` da resposta em memória, e não de um ficheiro. O `run_id` entra no fluxo como dado: é obtido com um `INSERT ... RETURNING` na tabela `execucao` e cruzado com as catorze mil linhas por um `Join rows`, produzindo o mesmo identificador em todas as linhas. As variáveis do Kettle

não são substituídas dentro de um `Add constants`, por isso a solução passou por tratar o identificador como campo e não como variável.

As três escritas ficam em série. Um passo com três setas de saída distribui as linhas pelos ramos em vez de as duplicar, e os três ficheiros do snapshot ficariam incompletos. Em série, cada escritor grava tudo e passa as linhas ao seguinte. O snapshot ocupa cerca de 16 MB nos três formatos.

Passo	Tipo	Papel
`Generate rows`	Generate rows	uma linha vazia, para alimentar o passo seguinte
`REST client`	REST client	um único pedido HTTP à DGEG
`JSON input`	JSON input	lê o campo `resultado` da resposta
`Criar execucao`	Table input	`INSERT ... RETURNING run_id` na tabela `execucao`
`Juntar run_id`	Join rows	cruza 1 linha com 14 213
`Limpar dummy`	Select values	remove o campo auxiliar
`Escrever CSV`, `Escrever XML`, `Escrever JSON`	Text, XML e JSON output	os três formatos do snapshot

4.2 E2: carregamento do staging

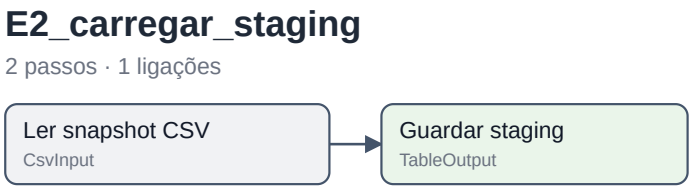


Figura 3. `E2_carregar_staging.ktr`: 2 passos, 1 ligação e nenhum acesso à rede.

A segunda transformação lê o CSV do snapshot e escreve na tabela `staging_preco`, que guarda o dado cru tal como a API o devolveu, em texto, incluindo o preço no formato original e as coordenadas sem conversão. Só `id_posto` e `run_id` são inteiros.

Manter o dado cru em texto é deliberado. A conversão de número do Kettle segue o locale da máquina, e num sistema em português o ponto é separador de milhares: no primeiro ensaio, 16 de 13 957 coordenadas saíram corrompidas, sem qualquer erro até ao momento da gravação. Guardar em texto e converter na fase seguinte, com formato explícito, elimina essa classe de erro.

4.3 T1: transformação

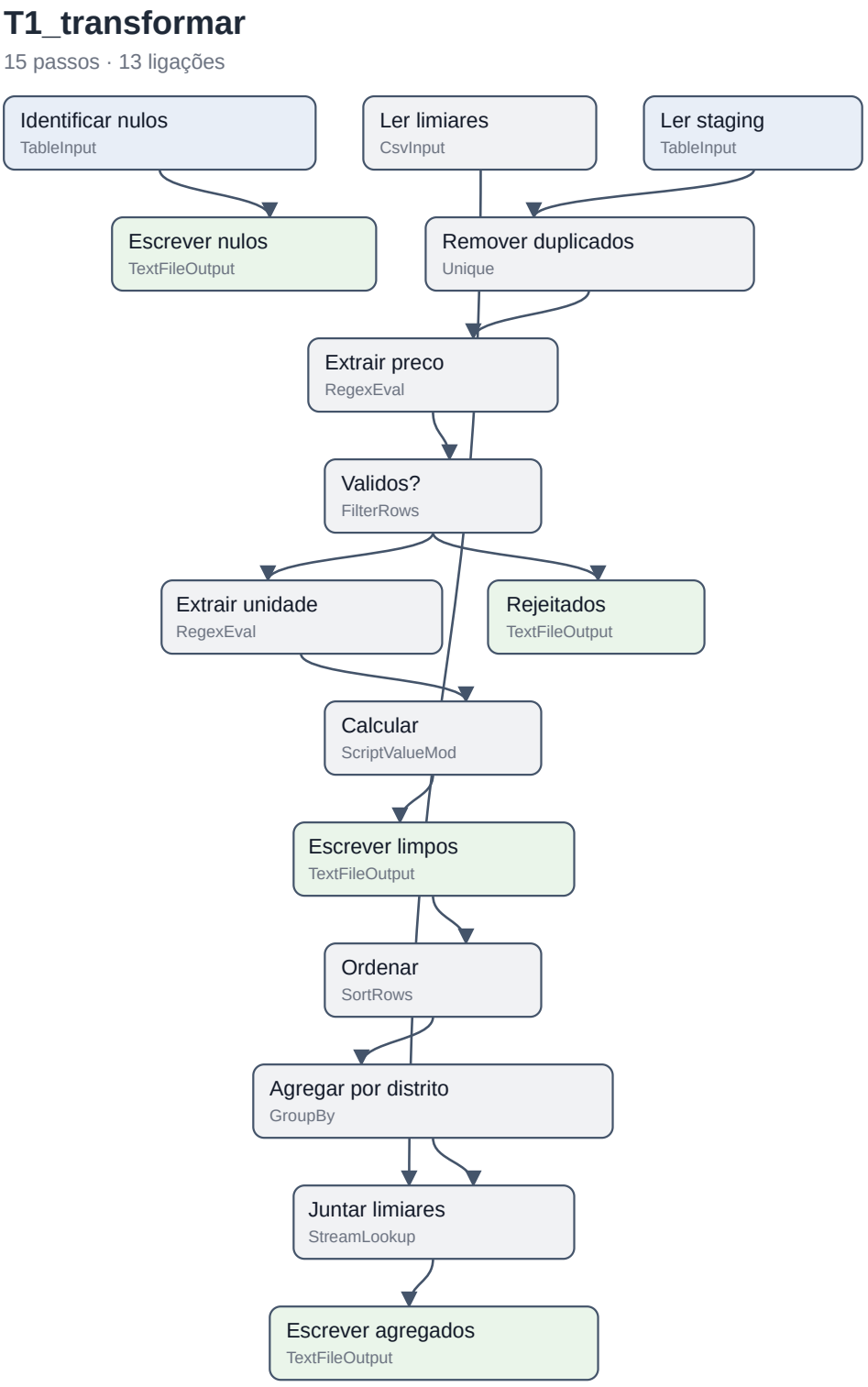


Figura 4. `T1_transformar.ktr`: 15 passos e 13 ligações. O ramo dos valores em falta corre em paralelo.

A transformação da Figura 4 lê o staging e aplica, por esta ordem, a remoção de duplicados por posto e combustível, a extração do preço com expressão regular, a filtragem dos preços válidos, a derivação da unidade a partir do nome do combustível, o cálculo do preço numérico e a normalização da marca. Em paralelo, um segundo caminho identifica os registos com campos em falta e escreve-os em `nulos.csv`.

Operação pedida no enunciado	Como está implementada
Expressões regulares	`Extrair preco` com `^([0-9]+),([0-9]{3}) €$` e `Extrair unidade` com `^.*€/(kg\|m3)$`
Identificação de valores nulos	`Identificar nulos`, por consulta com `coalesce(col,'')=''`, com ficheiro de saída próprio
Remoção de duplicados	`Remover duplicados`, com `Unique` pela chave `run_id`, `id_posto` e `combustivel`
Filtragem condicional	`Validos?`, com o ramo `Rejeitados` para o que não casa com o padrão do preço
Agrupamentos e cálculos	`Agregar por distrito` com contagem, média, mínimo e máximo, depois de `Ordenar`
Cruzamento com configuração	`Juntar limiares`, por lookup do preço-alvo do combustível
Mapeamento de esquema	texto do staging convertido para tipos, renomeações e unidade derivada

Três decisões mudam o resultado. A primeira é agregar sempre por combustível e nunca no conjunto, pela razão explicada na secção 2: com um alvo único para todos os grupos ficam 87 grupos abaixo do alvo, com o alvo certo por combustível ficam 2. A segunda é guardar o preço-alvo num ficheiro de configuração lido pelo processo, e não no código, para que a meta se possa mudar sem abrir a transformação. A terceira é manter as linhas com campos em falta em vez de as descartar. São 10 linhas, de dois postos de Viseu sem código postal, e a falta é do dado de origem: o preço existe e é válido para a análise, pelo que a linha fica no modelo e o problema fica registado no ficheiro de nulos.

A execução documentada leu 14 213 linhas, removeu zero duplicados, identificou 10 linhas com código postal em falta, produziu 208 grupos de distrito e combustível e encontrou 2 grupos abaixo do preço-alvo.

4.4 L1 a L4: modelo dimensional

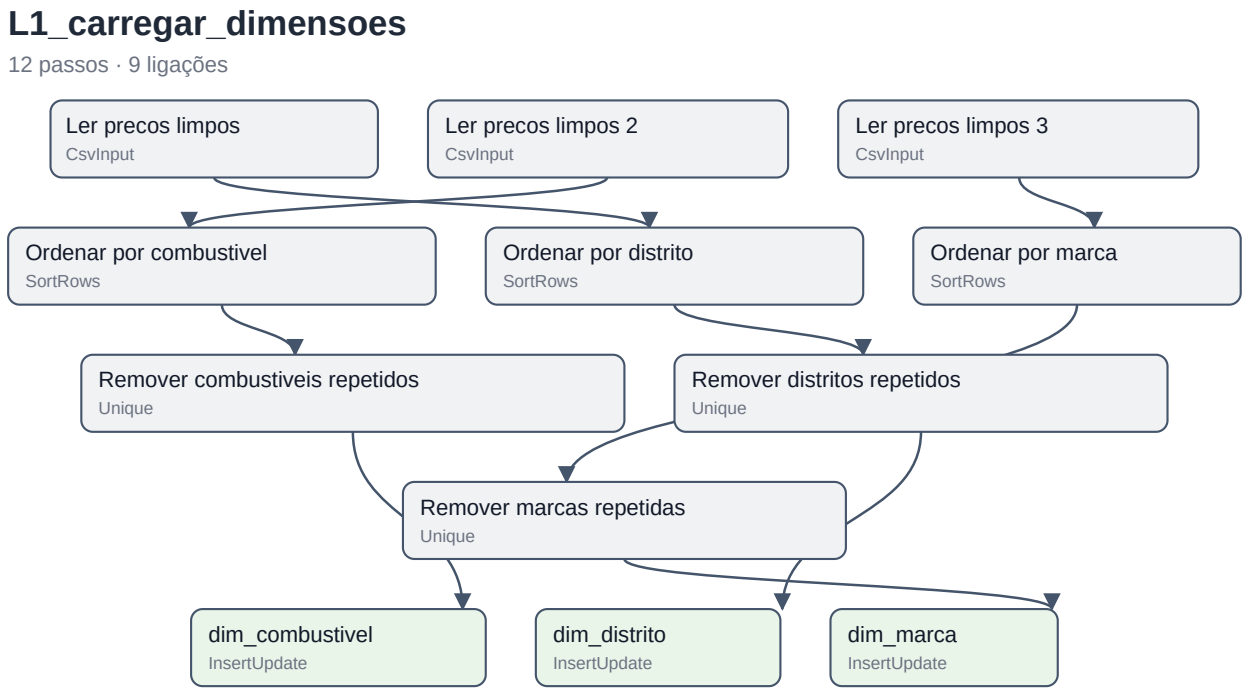


Figura 5. `L1_carregar_dimensoes.ktr` : dimensões de distrito, combustível e marca.

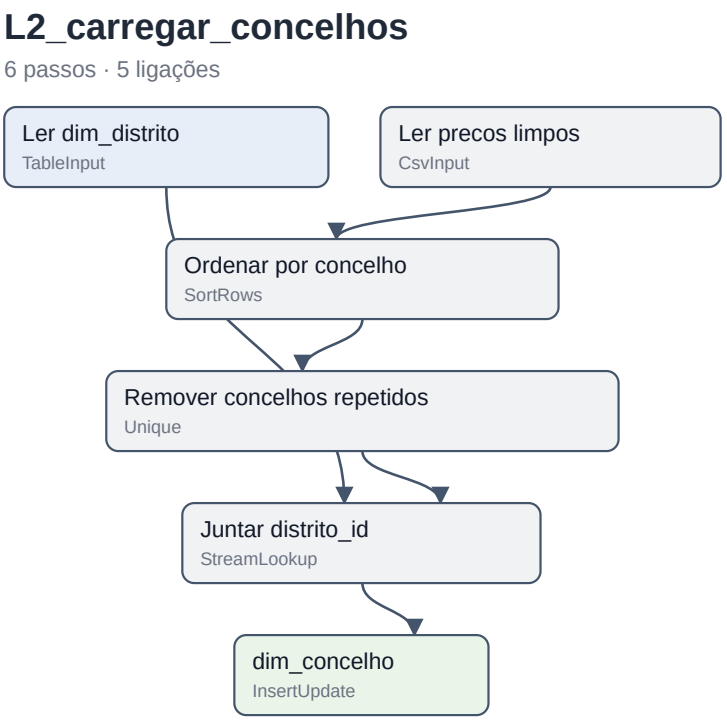


Figura 6. `L2_carregar_concelhos.ktr`: concelhos com a chave do distrito resolvida por lookup.

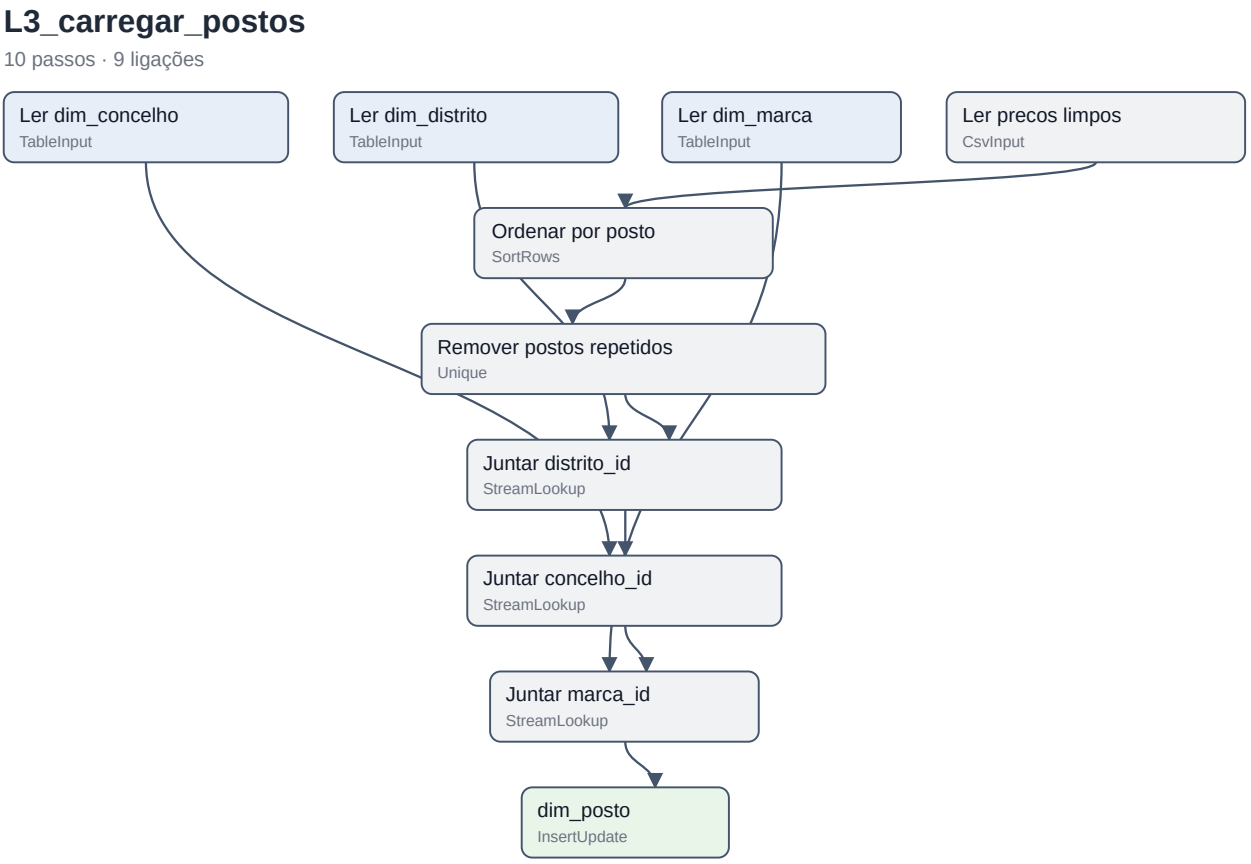


Figura 7. `L3_carregar_postos.ktr`: postos com as três chaves estrangeiras e as coordenadas já convertidas.

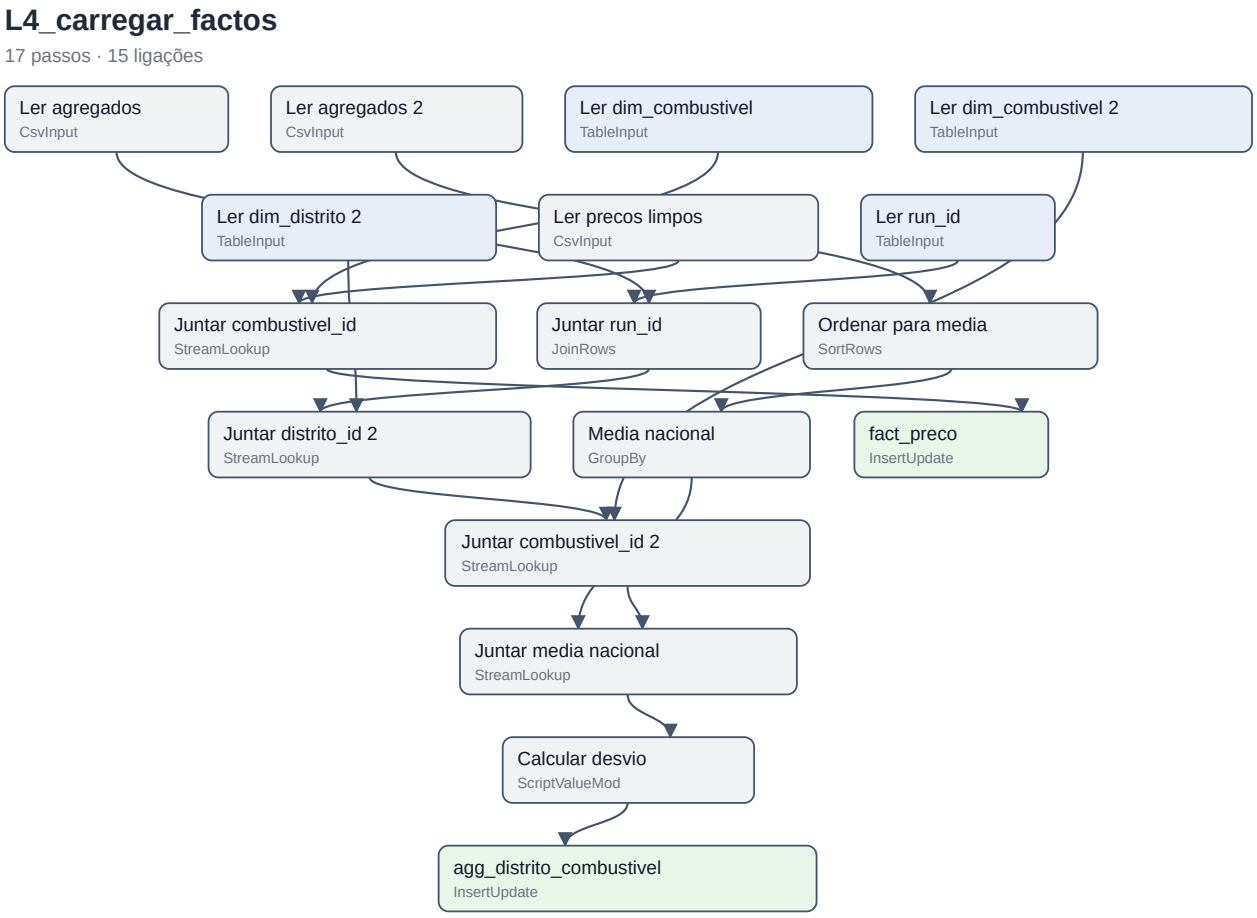


Figura 8. `L4_carregar_factos.ktr`: tabela de factos e agregados, com o desvio à média nacional calculado em SQL.

As quatro transformações das Figuras 5 a 8 carregam o modelo. As dimensões usam `Insert/Update` pela chave natural e não `Table output`. A diferença é importante quando o processo corre mais de uma vez: a chave natural do distrito é o nome, a do posto é o `Id` da DGEG e a do combustível é o nome com a unidade, pelo que a segunda execução atualiza em vez de duplicar. Uma escrita simples criaria uma dimensão nova a cada execução e as chaves estrangeiras dos factos deixariam de apontar para o registo certo.

A dimensão dos postos cresce com o tempo, porque o `Insert/Update` nunca apaga. Na execução documentada a dimensão tinha 3 253 linhas para 3 143 postos com preço no dia, ou seja 110 postos sem facto. São postos que saíram do ficheiro da DGEG entre execuções e que ficam na dimensão como histórico. É uma consequência aceitável do modelo e está assinalada na secção 7.

Tabela	Chave natural	Registos na execução documentada
`dim_distrito`	nome	18
`dim_combustivel`	nome e unidade	14
`dim_marca`	nome normalizado	55
`dim_concelho`	nome e distrito	277
`dim_posto`	`Id` da DGEG	3 143 por execução
`fact_preco`	`run_id`, posto e combustível	14 213
`agg_distrito_combustivel`	`run_id`, distrito e combustível	208

O desvio à média nacional é calculado em SQL, com uma janela por combustível, e não no Kettle. A média nacional tem de ser calculada sobre o conjunto dos distritos do mesmo combustível e o SQL exprime isso numa linha, sem passar os 208 grupos por uma ordenação e um segundo agrupamento.

4.5 L5 e L6: exportação e corpo do email

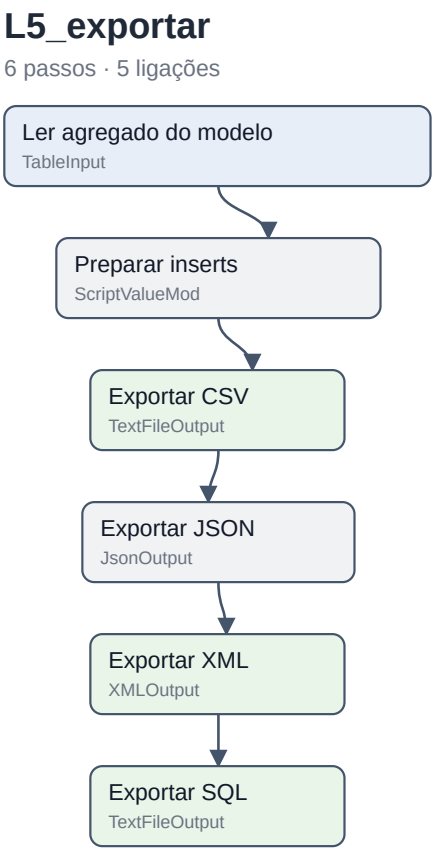


Figura 9. `L5_exportar.ktr`: quatro formatos de saída a partir do mesmo agregado.

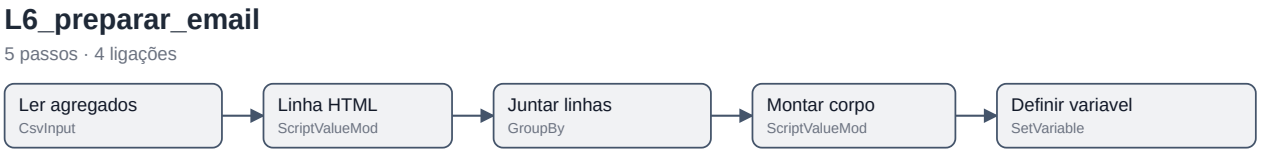


Figura 10. `L6_preparar_email.ktr`: monta o corpo HTML do email a partir do CSV de agregados.

Ficheiro exportado	Formato	Conteúdo
`agg_distrito_combustivel.xml`	XML	208 registos do agregado
`agg_distrito_combustivel.json`	JSON	208 registos
`agg_distrito_combustivel.csv`	CSV com separador `;`	208 registos e cabeçalho
`agg_distrito_combustivel.sql`	SQL	208 instruções `INSERT` prontas a executar
`relatorio.html`	HTML	relatório do dia gerado por XSLT sobre o XML

A exportação da Figura 9 escreve os quatro formatos. A folha XSLT, montada na Figura 10, é usada como conversão de formato dentro do job: lê o XML exportado e gera um relatório HTML com uma linha por grupo e os alertas destacados. É a técnica do exemplo do professor e cobre a conversão de XML para outro formato pedida no enunciado.

O corpo do email é construído na transformação `L6`, a partir do CSV de agregados, e guardado numa variável do ambiente do job. Ler o CSV e não a base de dados permite correr o job sem ligação ao PostgreSQL e mantém o email alinhado com os ficheiros exportados na mesma execução.

4.6 Dashboard

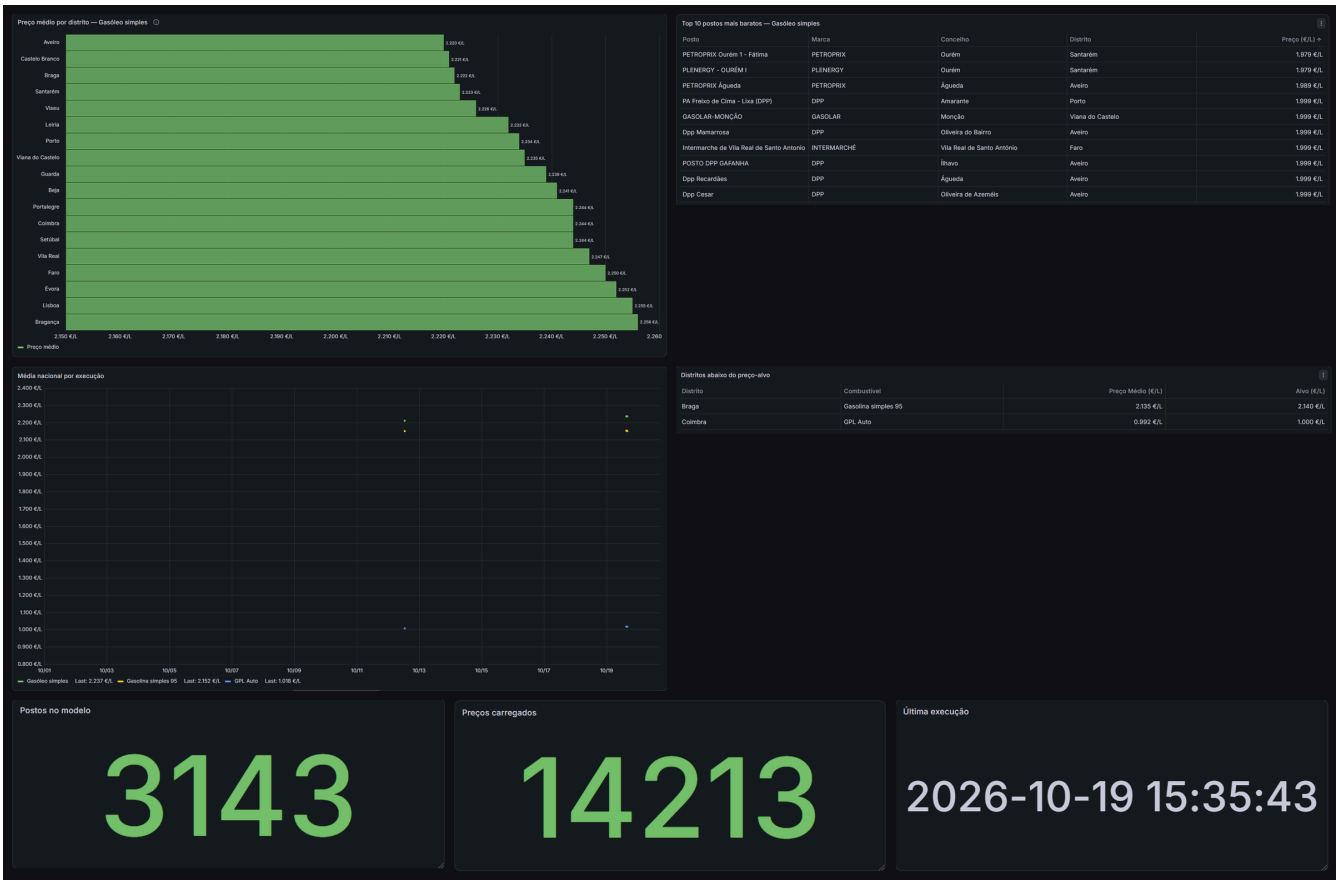


Figura 11. Painéis do dashboard no Grafana, sobre o modelo dimensional: preço médio por distrito, top 10 dos postos mais baratos, média nacional por execução, distritos abaixo do preço-alvo e os três indicadores da última execução. As capturas foram feitas painel a painel e reunidas numa só imagem.

O dashboard é a última peça da fase de carga. Lê as tabelas `agg_distrito_combustivel`, `fact_preco`, as dimensões e `execucao`, e não escreve nada na base: todas as consultas são `select`. A fonte de dados é uma ligação PostgreSQL configurada no Grafana com o mesmo utilizador que o Kettle usa.

Os quatro painéis e os três indicadores respondem às perguntas do problema: onde está o combustível mais barato no país, quanto cada distrito se afasta da média nacional, que grupos ficaram abaixo do preço-alvo e qual foi a última execução registada. Os indicadores contam pela tabela de factos da execução mais recente (`max(run_id)`) e não pela dimensão dos postos, porque as dimensões acumulam postos que já saíram do ficheiro da DGEG entre execuções.

O painel do preço médio por distrito tem o eixo dos preços a começar em 2,15 €/L e não em zero. É uma decisão consciente: os dezoito distritos vão de 2,220 a 2,256 €/L, uma amplitude de 1,6 %, e num eixo a começar em zero as barras ficariam todas do mesmo comprimento. A nota está na descrição do painel, para o gráfico não induzir em erro quem o lê.

O painel dos distritos abaixo do alvo compara cada grupo com o preço-alvo do mesmo ficheiro de limiares que a fase T usa e mostra apenas os grupos que ficaram por baixo. Na execução de 19/10 ficaram dois: Braga na gasolina 95, a 2,135 €/L contra um alvo de 2,140, e Coimbra no GPL, a 0,992 contra 1,000.

4.7 Recolha contínua: Node-RED e MQTT

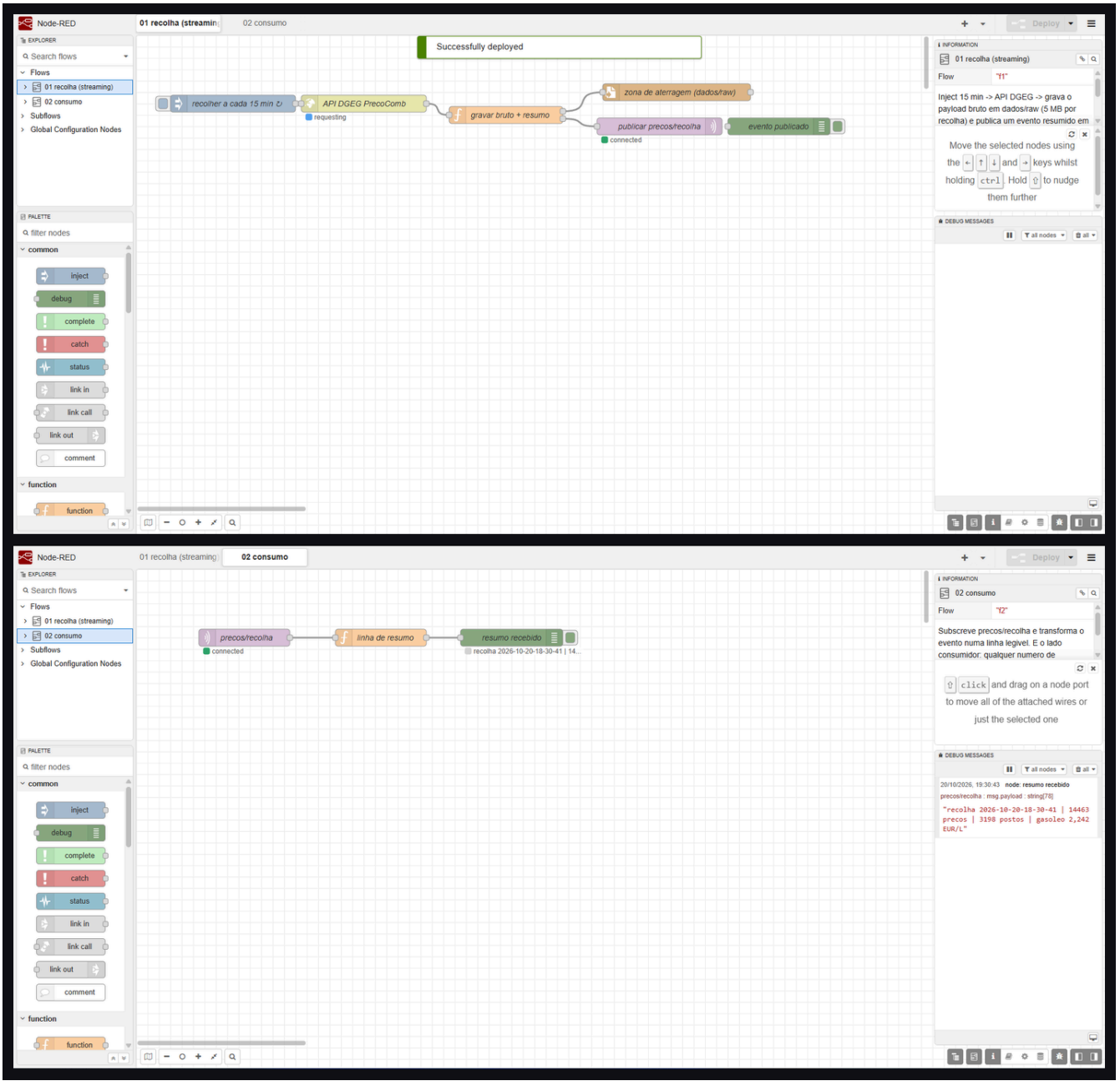


Figura 12. Os dois fluxos no Node-RED: em cima, a recolha automática que grava o payload e publica o evento em `precos/recolha`; em baixo, o consumidor subscrito ao mesmo tópico. No canto inferior direito do segundo fluxo lê-se a mensagem recebida na execução de 20/10.

A recolha da fase E é manual: alguém abre o Spoon e prime Play. Para o processo poder correr sozinho, ficou também um fluxo no Node-RED que faz a mesma chamada à API a cada quinze minutos, sem ninguém a tocar.

O fluxo tem cinco nós e um de debug. O `inject` é o relógio. O `http request` chama o mesmo endpoint da E1. A função seguinte faz duas coisas com a resposta: grava o payload em bruto na zona de aterragem (`dados/raw/precos-<data>.json`) e publica um evento de seis campos no tópico

`precos/recolha` do broker Mosquitto, que corre na própria máquina. Do outro lado, um segundo fluxo subscreve esse tópico e transforma o evento numa linha legível.

O evento não leva o payload completo de propósito. Uma recolha são 14 463 registos, 5,3 MB: um broker serve para eventos pequenos e não para transportar ficheiros, por isso o volume fica na zona de aterragem e pelo MQTT passa só o resumo (número de registos, número de postos, preços de gasóleo e média). A execução de 20/10 às 18:30 recolheu 14 463 preços de 3 198 postos, com 2 957 registos de gasóleo simples e uma média de 2,242 €/L.

Este fluxo não substitui o job: é uma segunda porta de entrada para o mesmo dado, e é isso que mostra. O produtor não sabe quem consome, o consumidor não sabe quem recolheu, e qualquer número de processos pode subscrever o mesmo tópico. No enunciado, o Node-RED aparece como exemplo de fonte em *streaming* e como ferramenta complementar; é o critério de mais-valia que este trabalho ainda não cobria.

5. Jobs

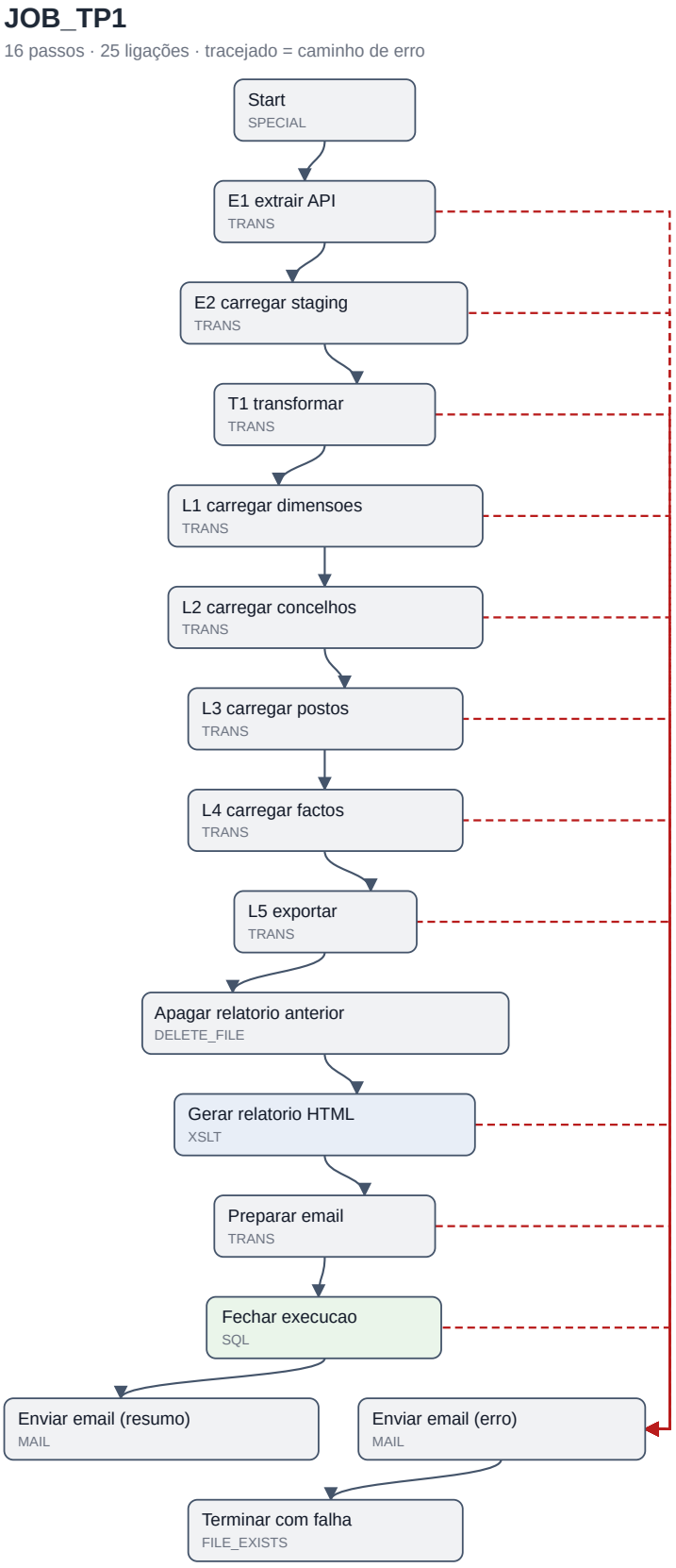


Figura 13. `JOB_TP1.kjb`: 16 entradas e 25 ligações. A tracejado vermelho, o caminho de erro.

#	Entrada	Tipo	Função
1	Start	Special	início da execução
2	E1 extrair API	Transformation	extração da API
3	E2 carregar staging	Transformation	carga do dado cru
4	T1 transformar	Transformation	limpeza, cálculo e agregação
5	L1 carregar dimensoes	Transformation	dimensões
6	L2 carregar concelhos	Transformation	concelhos
7	L3 carregar postos	Transformation	postos
8	L4 carregar factos	Transformation	factos e agregados
9	L5 exportar	Transformation	quatro exportações
10	Apagar relatorio anterior	Delete files	remove o HTML da execução anterior
11	Gerar relatorio HTML	XSL transformation	XML e folha XSL para HTML
12	Preparar email	Transformation	constrói o corpo HTML do email
13	Fechar execucao	SQL	fecha a linha da tabela `execucao`
14	Enviar email (resumo)	Mail	resumo da execução
15	Enviar email (erro)	Mail	aviso de falha
16	Terminar com falha	File exists	devolve código de erro ao sistema

No caminho feliz, representado na Figura 13, as entradas correm pela ordem da tabela. A execução documentada demorou cerca de dez segundos, das 14:35:43 às 14:35:53, e terminou com todas as entradas a devolver verdadeiro.

O caminho de erro é feito de ligações do tipo não avaliar e não incondicional, que o Kettle usa precisamente para o ramo de falha. Qualquer uma das onze fases com ligação de erro desvia para a entrada 15, que envia o email de falha, e daí para a entrada 16. Essa última entrada é um `File exists` sobre um marcador deixado pela falha: se o marcador existe, a entrada devolve falso e o job termina com código de erro em vez de terminar em silêncio. Foi o que permitiu testar o caminho de erro com a primeira fase deliberadamente quebrada, obtendo código de retorno 1 e o email de erro enviado.

A entrada 13 fecha a linha da tabela `execucao` com o estado e as contagens. Na execução documentada ficaram registadas 14 213 linhas lidas e 14 213 linhas carregadas. Uma limitação conhecida: no ramo de erro o email é enviado mas a execução não é fechada, e fica em `a correr`. Está assinalada na secção 7.

Configuração fora do repositório

Nenhuma credencial está no repositório. O job lê quatro variáveis do ficheiro `kettle.properties` da máquina onde corre:

Variável	Para que serve
`DB_PASSWORD`	password do PostgreSQL
`MAIL_TO`	destinatário do email de resumo e de erro
`MAIL_USER`	conta de envio
`MAIL_PASSWORD`	password de aplicação dessa conta

O servidor de email está configurado com ligação segura na porta 587. A password usada é uma password de aplicação criada para este efeito, que pode ser revogada sem alterar a password da conta. O Kettle lê este ficheiro uma única vez, no arranque, pelo que uma alteração às variáveis obriga a reiniciar a ferramenta.

6. Vídeo com demonstração

O vídeo percorre o processo pela ordem em que corre: o relatório, as transformações no Spoon com os passos a verde e o log de execução, os ficheiros de `dados/export/`, o relatório HTML gerado por XSLT a partir do XML, a linha da tabela `execucao` da execução do dia no pgAdmin, o dashboard com os quatro painéis e os três indicadores, o fluxo do Node-RED com o evento a chegar ao subscritor e o repositório no GitHub. A caixa de correio não aparece na gravação; o email é enviado pelo job e está descrito na secção 5.

A gravação é de 20/10, numa execução posterior à que este relatório documenta: 14 463 preços de 3 198 postos, 208 grupos e três grupos abaixo do preço-alvo (Braga e Aveiro na gasolina 95 e Coimbra no GPL). Os números não coincidem com os do texto porque os preços são publicados de novo todos os dias, e é por isso que cada execução fica datada na tabela `execucao`.

Link	https://youtu.be/bAerpskohRc
Duração	5 min 16 s

Figura 14. QR Code para o vídeo de demonstração, no YouTube como não listado.

7. Conclusão e trabalhos futuros

O trabalho cumpriu as três fases pedidas. A extração faz uma chamada à API e produz o snapshot em três formatos, o carregamento para o staging não usa rede, a transformação limpa e agrega, e a carga escreve o modelo dimensional e exporta em quatro formatos. A execução documentada neste relatório processou 14 213 preços de 3 143 postos, produziu 208 grupos de distrito e combustível, identificou 2 grupos abaixo do preço-alvo, ficou registada na tabela `execucao` com as contagens certas e deixou o dashboard com quatro painéis e três indicadores.

Além do job, ficou uma segunda via de recolha em Node-RED, que faz a mesma chamada de forma automática e anuncia cada recolha por MQTT. Não substitui o job: mostra que o mesmo dado pode chegar por um fluxo contínuo e ser consumido por processos que não conhecem o produtor.

A parte que deu mais trabalho não foi a que parecia difícil. A chamada à API é uma linha e demora três segundos. O tempo foi gasto nas armadilhas silenciosas da ferramenta: o `REST client` que devolve zero linhas sem erro quando não tem entrada, o `JSON input` configurado para ler de ficheiro que não lê nada, o `Group by` sem `Sort rows` que agrupa chaves não consecutivas em grupos separados, o `Filter rows` com o ramo verdadeiro vazio que engole as linhas todas e o ficheiro CSV que não voltava a ler-se por causa das moradas com quebras de linha. Nenhum destes erros produz mensagem. Todos produzem um resultado plausível e errado, e a defesa foi comparar contagens entre fases em cada execução.

Se repetisse o trabalho, começaria por definir essas contagens esperadas antes de construir as transformações. Foi o que acabei por fazer, e foi o que permitiu apanhar os erros acima.

Limitações identificadas e trabalho futuro

Limitação	Detalhe	Correção proposta
Instante da execução sem fuso horário	`iniciada_em` é `timestamp without time zone`, pelo que o instante depende do fuso do servidor	passar a coluna a `timestamptz`
O ramo de erro não fecha a execução	uma execução que falha fica em `a correr`, com zero linhas	fechar a execução no caminho de erro, com estado de falha
Dimensão dos postos acumula	3 253 linhas na dimensão contra 3 143 postos com preço na execução de 19/10	marcar inatividade com uma coluna própria em vez de manter sem distinção
Pasta do snapshot definida manualmente	o parâmetro tem por omissão a data da última execução	derivar a pasta da data da execução
Entrada de remoção do relatório sem ligação de erro	se a remoção falhar, o job continua e o HTML é reescrito de qualquer forma	acrescentar a ligação de erro, por coerência
Sem transferência para servidor remoto	FTP e SFTP são opcionais no enunciado	acrescentar um passo de envio dos ficheiros
Broker MQTT sem autenticação	o Mosquitto escuta apenas em `127.0.0.1` e aceita ligações anónimas, o que serve a demonstração local mas não uma máquina exposta	`password_file` e TLS antes de sair da máquina

8. Referências

- DGEG, *Preços de Combustíveis Online*, API PrecoComb. https://precoscombustiveis.dgeg.gov.pt/api/PrecoComb/PesquisarPostos
- Hitachi Vantara, *Pentaho Data Integration Documentation*, versão 10.2. https://help.hitachivantara.com/Documentation/Pentaho/10.2
- PostgreSQL Global Development Group, *PostgreSQL 17 Documentation*. https://www.postgresql.org/docs/17/
- Grafana Labs, *Grafana Documentation*. https://grafana.com/docs/grafana/latest/
- OpenJS Foundation, *Node-RED User Guide*. https://nodered.org/docs/user-guide/
- Eclipse Foundation, *Eclipse Mosquitto Documentation*. https://mosquitto.org/documentation/
- pgAdmin Development Team, *pgAdmin 4 Documentation*. https://www.pgadmin.org/docs/
- W3C, *XSL Transformations (XSLT) Version 1.0*. https://www.w3.org/TR/xslt
- Google, *Sign in with app passwords*, Apoio ao utilizador da Conta Google. https://support.google.com/accounts/answer/185833
- Enunciado do Trabalho Prático I, Integração de Sistemas de Informação, 2026/2027.